Manufacturing Operational Insights Report

on

Marketing Planner Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: GAI-26

Industry Mentor Name: Mr. Ashwin

University Mentor Name: Prop Maya Yadav Baniya

Academic Year: 2025-2026

Table of Contents

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

Marketing teams and professionals frequently struggle to translate high-level business goals into structured, actionable execution plans. The manual process of breaking down campaigns into subtasks, validating resources, estimating costs, and scheduling dependencies is time-consuming, error-prone, and inconsistent. There is a need for an intelligent, automated solution that can take a natural language marketing goal and output a validated, sequenced execution plan ready for immediate team execution.

2. Project Objectives

- Build an AI-powered full-stack application that decomposes natural language marketing goals into structured, validated subtasks.
- Leverage the LLaMA 3.3 large language model via the Groq inference API for intelligent goal decomposition.
- Implement a multi-agent pipeline (Planner → Validator → Scheduler) using the CrewAI framework.
- Validate each subtask against budget constraints, team availability, and timeline feasibility using dedicated validation tools.
- Provide a responsive, authenticated web interface built on React and TypeScript for seamless user interaction.
- Return an ordered execution schedule with dependency tracking, flagged tasks, and a comprehensive summary.

3. Scope of the Project

This project covers the end-to-end development of the Marketing Planner Agent, including:

- Full-stack web application with a React/TypeScript frontend and Python FastAPI backend.

- Firebase Authentication supporting Google, GitHub, and Email/Password sign-in.
- A multi-agent AI pipeline using CrewAI and Groq API for intelligent plan generation.
- Three validation tools (Budget Checker, Team Availability, Timeline Validator) with mock data designed for production replacement.
- Deployment on Vercel (frontend) and Render (backend) cloud platforms.

The project does not cover infrastructure provisioning, CI/CD pipeline configuration, persistent database storage, or advanced features such as vector database integration.

Proposed Solution

1. Key Features

- Natural Language Goal Input: Users submit high-level marketing goals in plain English via a simple textarea interface.
- Multi-Agent AI Pipeline: Three specialized CrewAI agents — Planner, Validator, and Scheduler — work sequentially to produce a validated execution plan.
- LLM-Powered Task Decomposition: The Planner Agent uses LLaMA 3.3-70b via Groq API to generate 5-7 structured subtasks with cost, team, and timeline estimates.
- Automated Resource Validation: Each subtask is validated against budget limits, team availability, and timeline conflicts using Python-based validation tools.
- Dependency-Aware Scheduling: The Execution Scheduler resolves task dependencies and produces an ordered timeline sorted by start day.
- Firebase Authentication: Supports Google, GitHub, and Email/Password sign-in with client-side route protection.
- Interactive Timeline UI: The frontend renders the final schedule as a visual timeline with statistics, flagged tasks, and a project summary panel.

- Dark/Light Theme Toggle: User preference persisted across sessions via localStorage.

2. Overall Architecture / Workflow

The application uses a three-tier architecture with clear separation between presentation, business logic, and external services.

2.1 System Overview

Layer	Technology	Responsibility	Hosting
Presentation	React 18, TypeScript, Tailwind CSS, Vite	User interface, routing, authentication UI	Vercel
API Gateway	FastAPI, Uvicorn, Python 3.11	HTTP endpoints, request validation, CORS	Render
Agent Layer	CrewAI, Groq API, LLaMA 3.3-70b	Goal decomposition, multi-agent pipeline	Render
Validation Layer	Python functions	Budget, team, timeline validation	Render
Scheduling Layer	Python functions	Dependency resolution, schedule generation	Render
Authentication	Firebase Authentication	Google, GitHub, Email/Password sign-in	Firebase Cloud

The system follows a client-server model. The React frontend communicates exclusively with the FastAPI backend via a single REST endpoint (POST /api/plan). All AI

processing, validation, and scheduling logic resides on the backend. The frontend manages authentication state via Firebase and is responsible only for rendering results.

Plan Generation Flow: The user submits a marketing goal → A POST request is sent to FastAPI → The Planner Agent calls Groq (LLaMA 3.3-70b) to generate structured subtasks → The raw LLM text is parsed line-by-line (fault-tolerant) → Each task is validated sequentially by Budget, Team Availability, and Timeline tools → Validated tasks are sorted and dependencies resolved by the Execution Scheduler → The final JSON schedule is returned and rendered as an interactive timeline.

2.2 Application Design

The application uses a three-tier system with clear separation between presentation, logic, and external services.

- Frontend: A single-page application (SPA) with three primary routes: / (Landing), /login (Auth), and /planner (Protected).
- Backend: A FastAPI application exposing two HTTP endpoints that orchestrate the agent and tools in a sequential pipeline.

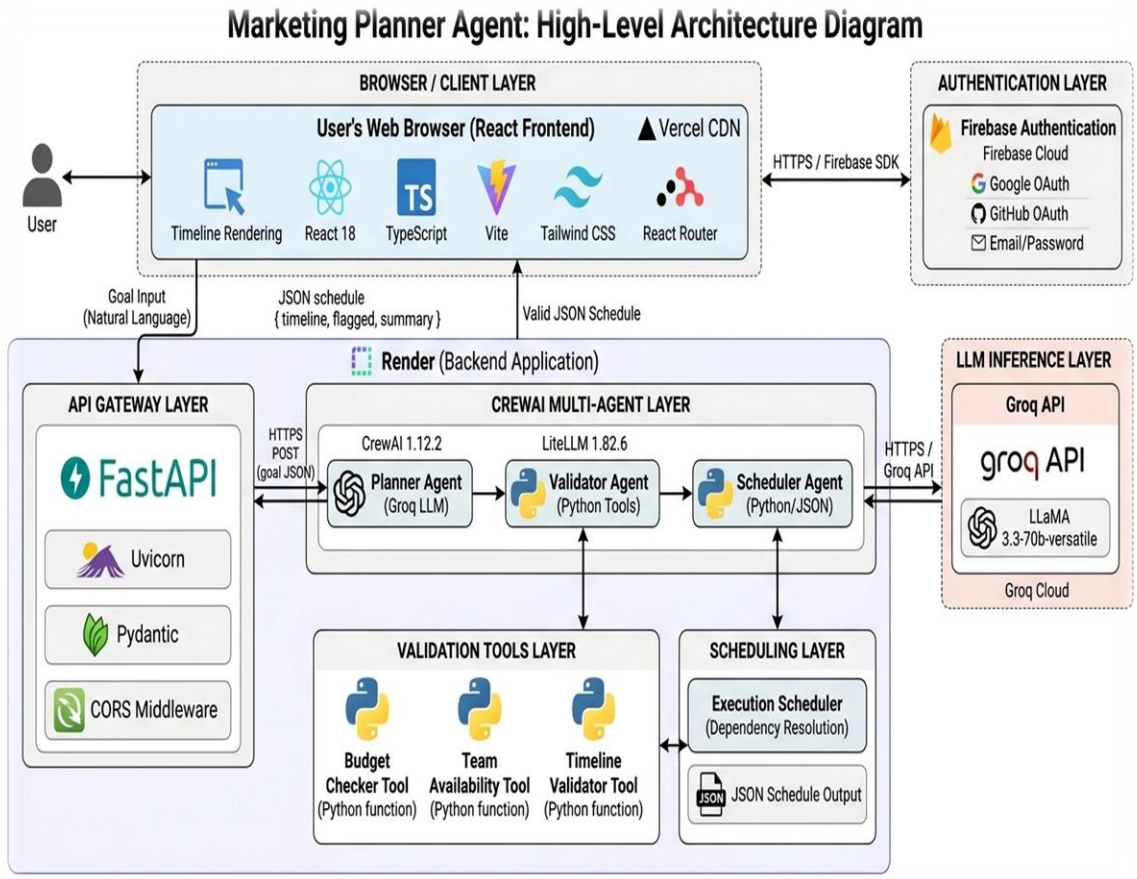


Fig.1: Architecture Diagram

2.3 Process Flow

2.3.1. Authentication Flow

Users authenticate via Firebase using Google, GitHub, or Email. Upon success, the user's email is stored in local Storage as auth_user, which the /planner route checks to allow access. The authentication sequence is as follows:

Step	Actor	Action	Result
1	User	Visits landing page at /	LandingPage renders with Log in and Sign up buttons
2	User	Clicks Continue with Google / GitHub	Firebase signInWithPopup called with respective AuthProvider
3	Firebase	Opens OAuth popup	User selects account
4	OAuth Provider	Returns authorization token to Firebase	Firebase exchanges token for UserCredential
5	LoginPage	Receives UserCredential	Stores email, name, photoURL in localStorage
6	React Router	Navigates to /planner	PlannerPage renders, reads auth_user from localStorage

2.3.2. Plan Generation Flow

1. Input: User submits a marketing goal via the PlannerPanel.
2. Request: A POST request is sent to the FastAPI backend.
3. Agent: The agent calls Groq (LLaMA 3.3) to generate structured subtasks.
4. Parsing: The raw text is parsed line-by-line using specific prefixes (TASK, COST, etc.).
5. Validation: Each task is checked sequentially against Budget, Team, and Timeline tools.
6. Scheduling: Validated tasks are sorted by start day and dependencies are resolved.
7. Output: The final JSON schedule is rendered as a timeline on the frontend.

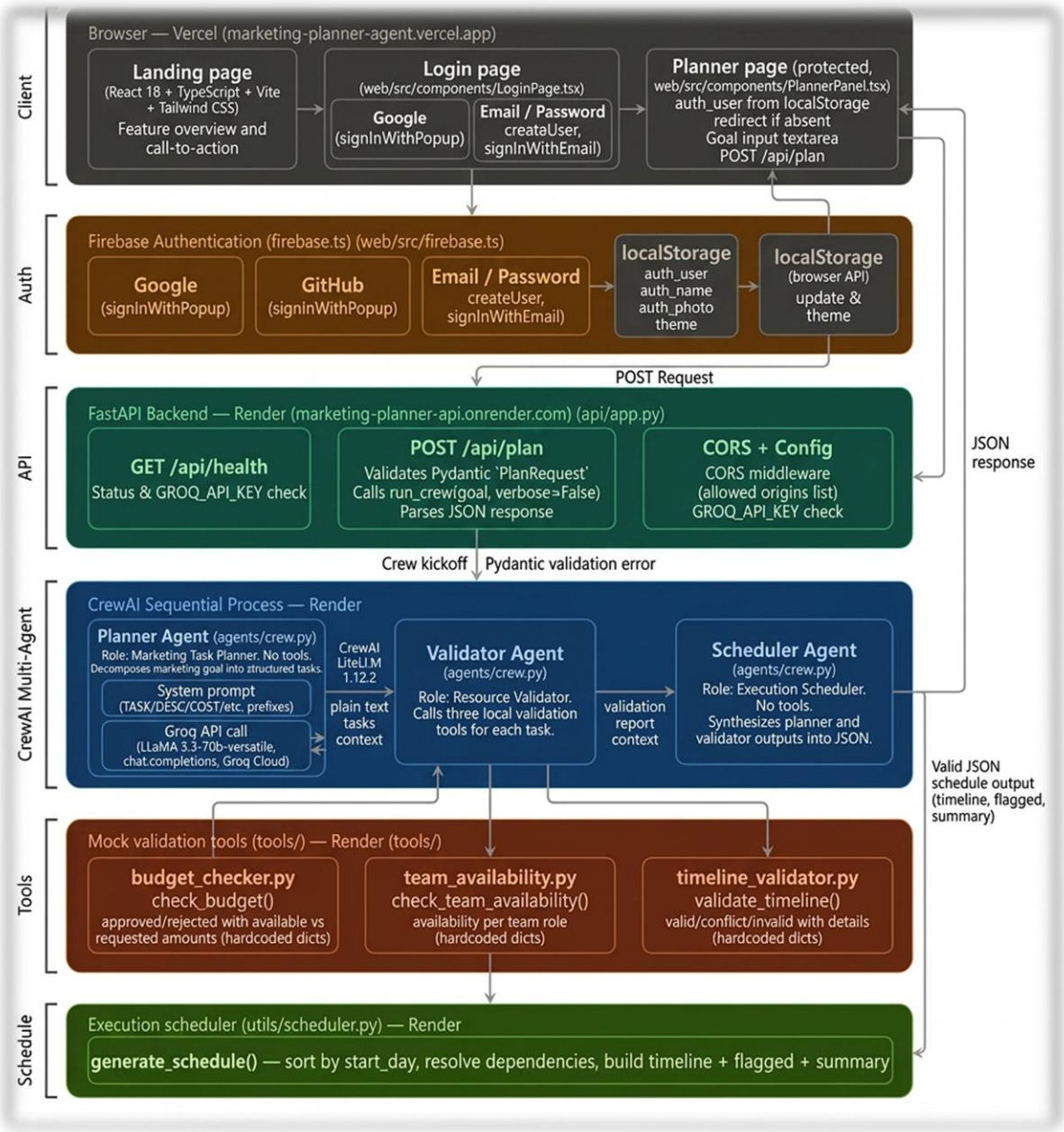


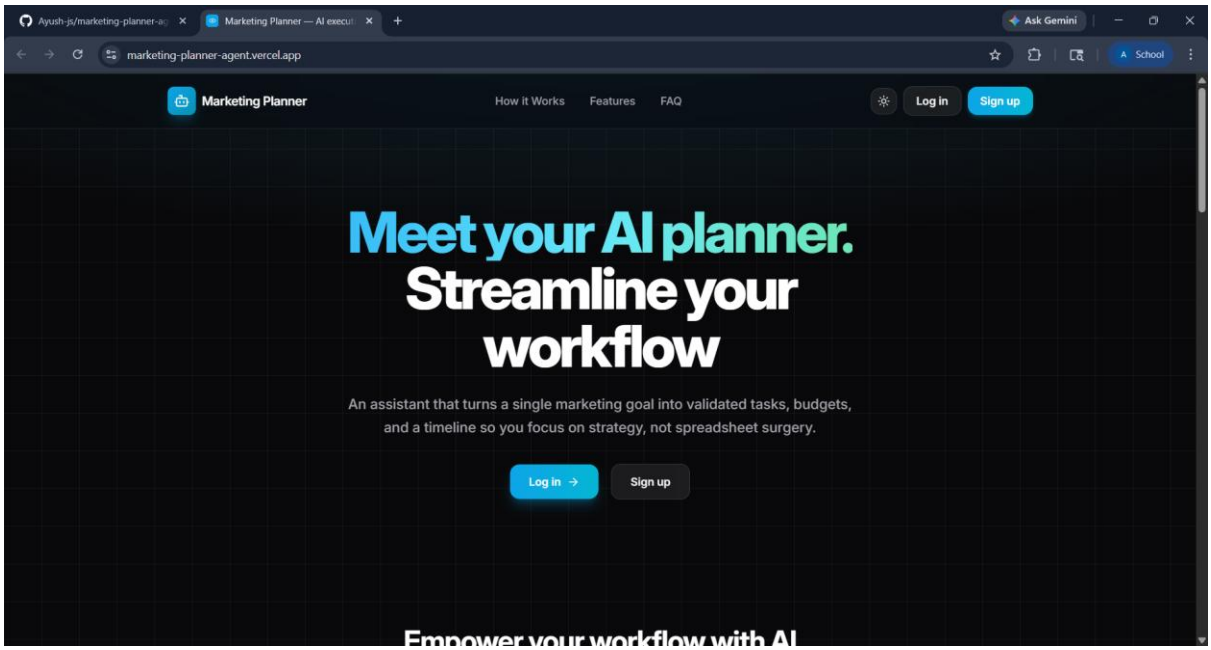
Fig.:2 Process flow

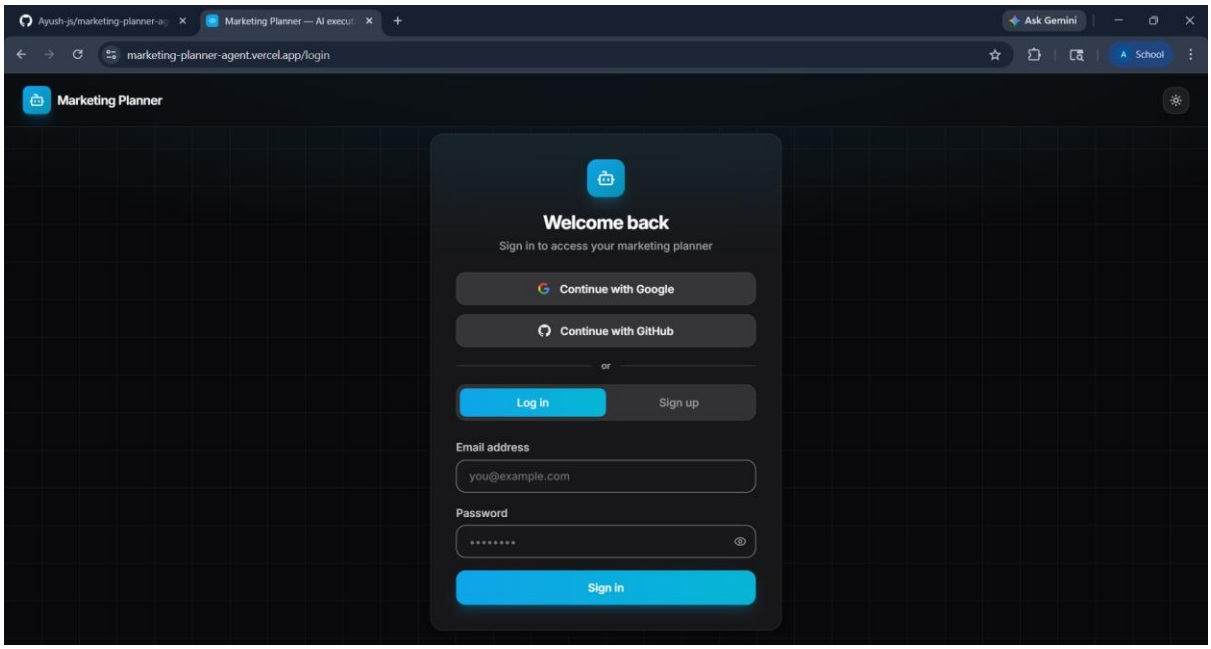
3. Tools & Technologies Used

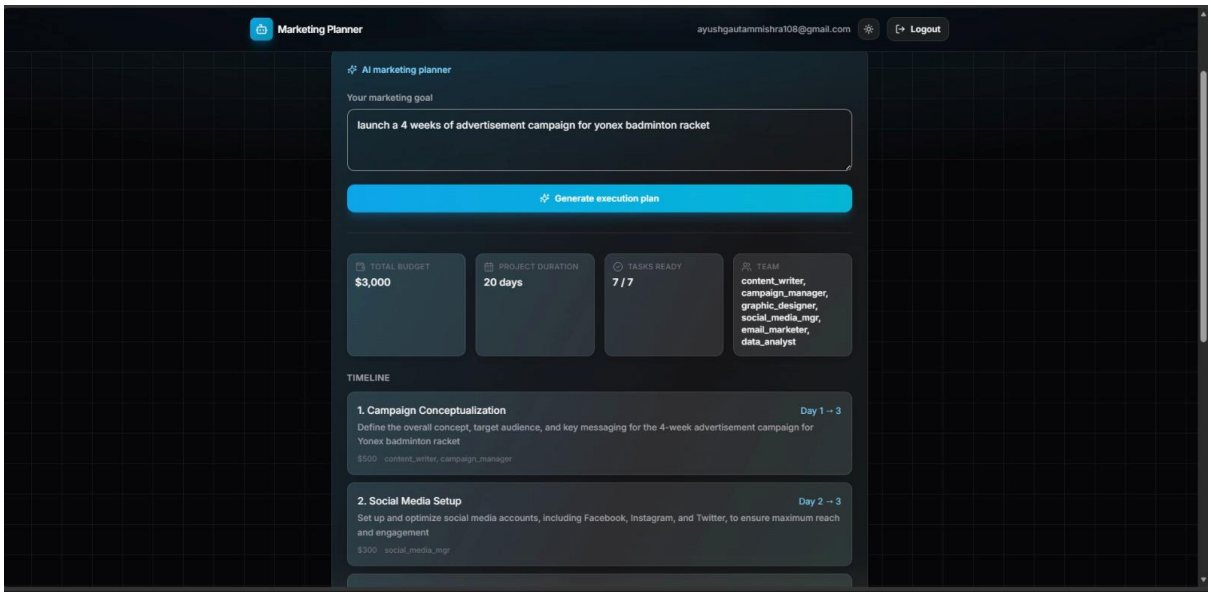
Category	Technology	Purpose
Frontend	React 18, TypeScript	UI framework with type safety
Frontend	Tailwind CSS	Utility-first styling
Frontend	Vite	Build tool and dev server
Frontend	React Router	Client-side routing
Backend	Python 3.11, FastAPI	REST API server
Backend	Uvicorn	ASGI server
Backend	Pydantic	Request/response validation
AI / Agents	CrewAI	Multi-agent orchestration
AI / Agents	LiteLLM	LLM provider routing
AI / Agents	Groq API	LLM inference (LLaMA 3.3-70b)
Authentication	Firebase Authentication	OAuth 2.0 / Email sign-in
Hosting	Vercel	Frontend deployment (CDN)
Hosting	Render	Backend deployment

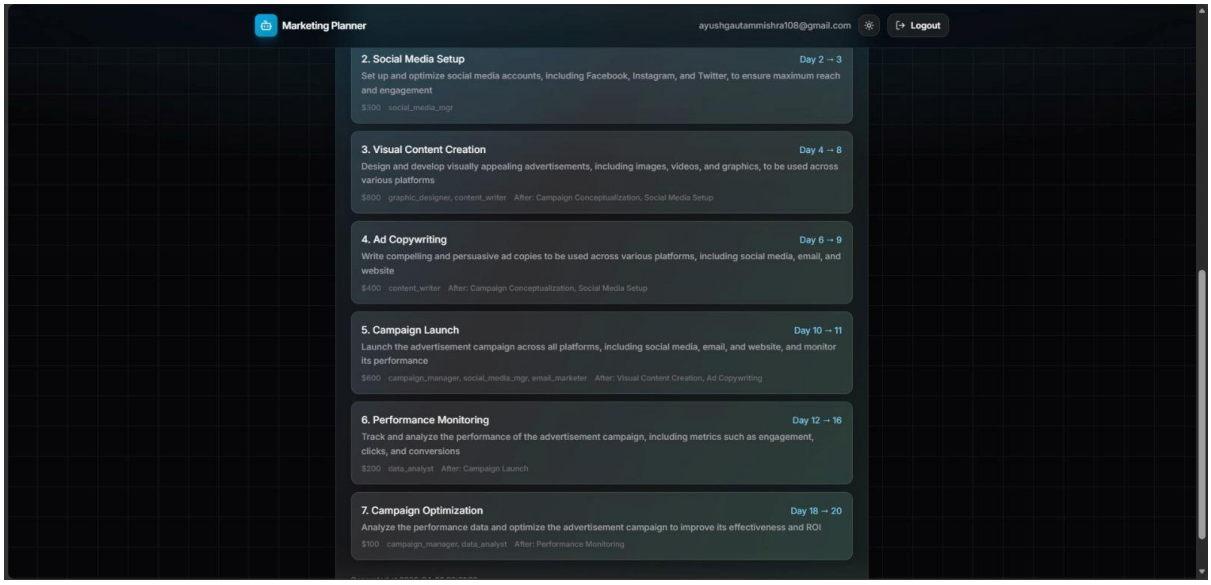
Results & Output

1. *Screenshots / outputs*






Marketing Planner
ayushgautammishra108@gmail.com
Logout
AI marketing planner
Your marketing goal
launch a 4 weeks of advertisement campaign for yonex badminton racket
Generate execution plan
TOTAL BUDGET
$3,000
PROJECT DURATION
20 days
TASKS READY
7 / 7
TEAM
content_writer, campaign_manager, graphic_designer, social_media_mgr, email_marketer, data_analyst
TIMELINE
1. Campaign Conceptualization
Day 1 → 3
Define the overall concept, target audience, and key messaging for the 4-week advertisement campaign for Yonex badminton racket
$500 content_writer, campaign_manager
2. Social Media Setup
Day 2 → 3
Set up and optimize social media accounts, including Facebook, Instagram, and Twitter, to ensure maximum reach and engagement
$300 social_media_mgr


Marketing Planner
ayushgautammishra108@gmail.com
Logout
2. Social Media Setup
Day 2 → 3
Set up and optimize social media accounts, including Facebook, Instagram, and Twitter, to ensure maximum reach and engagement
$300 social_media_mgr
3. Visual Content Creation
Day 4 → 8
Design and develop visually appealing advertisements, including images, videos, and graphics, to be used across various platforms
$800 graphic_designer, content_writer After: Campaign Conceptualization, Social Media Setup
4. Ad Copywriting
Day 6 → 9
Write compelling and persuasive ad copies to be used across various platforms, including social media, email, and website
$400 content_writer After: Campaign Conceptualization, Social Media Setup
5. Campaign Launch
Day 10 → 11
Launch the advertisement campaign across all platforms, including social media, email, and website, and monitor its performance
$600 campaign_manager, social_media_mgr, email_marketer After: Visual Content Creation, Ad Copywriting
6. Performance Monitoring
Day 12 → 16
Track and analyze the performance of the advertisement campaign, including metrics such as engagement, clicks, and conversions
$200 data_analyst After: Campaign Launch
7. Campaign Optimization
Day 18 → 20
Analyze the performance data and optimize the advertisement campaign to improve its effectiveness and ROI
$100 campaign_manager, data_analyst After: Performance Monitoring

2. Reports / dashboards / models

1. Reports and Dashboards

The application utilizes a specialized UI to present generated data to the user:

- Interactive Timeline Dashboard: The React frontend renders a visual timeline of tasks based on the order and start day provided in the API response.
- Validation Reports: The Validator Agent generates reports for each task, using an IS_VALID flag to indicate if a task meets budget, team, and timeline constraints.
- Flagged Tasks List: A dedicated section of the dashboard displays tasks that failed validation, providing the specific reasons for the conflict or rejection.
- Summary Statistics: The interface includes "stat boxes" that aggregate key metrics such as total budget, team members involved, and project duration.

2. Data Models

The system relies on several transient data structures to manage the flow of information:

- Schedule Data Model: The primary API response containing a timestamp (generated_at), task counts, the ordered timeline array, and the summary object.
- Schedule Task Model: Defines the properties of an individual task, including task_name, estimated_cost, team_members, and dependency tracking through depends_on.
- Authentication State Model: Manages user identity in the browser's localStorage, storing the user's email (auth_user), name, and photo URL retrieved from Firebase.
- Mock Validation Model: Uses in-memory Python dictionaries to simulate external databases for budget (available_budgets), team schedules (team_schedule), and existing timelines (scheduled_tasks).

3. Agentic Models

The "brains" of the system consist of three specialized LLM-driven agents:

- Planner Agent: Decomposes high-level goals into 5–7 structured subtasks using LLaMA 3.3 reasoning.
- Validator Agent: Acts as a resource checker by executing specific Python tools against the tasks generated by the planner.
- Scheduler Agent: Synthesizes the outputs of the previous two agents into a finalized, machine-readable JSON execution schedule.

4. Key Outcomes

- Successfully implemented a fully functional multi-agent AI pipeline capable of converting natural language marketing goals into structured, dependency-aware execution schedules.
- The Planner Agent reliably generates 5-7 subtasks per goal using a fault-tolerant line-by-line prefix parser (TASK/DESC/COST/TEAM/START/DURATION) instead of brittle JSON output.
- All three validation tools (Budget Checker, Team Availability, Timeline Validator) integrate seamlessly with the CrewAI Validator Agent, flagging tasks that exceed constraints.

- The React frontend renders interactive timelines with stat boxes, flagged task lists, and project summaries, achieving First Contentful Paint under 1 second and Time to Interactive under 2 seconds.
- End-to-end plan generation time is 8-15 seconds, dominated by Groq API inference time (3-6 seconds for the Planner Agent).
- Firebase Authentication successfully supports Google, GitHub, and Email/Password sign-in with client-side route protection.

- The application is live and accessible at https://marketing-planner-agent.vercel.app.

Conclusion

The Marketing Planner Agent successfully demonstrates the core principles of Agentic AI — autonomous goal decomposition, tool use, multi-agent collaboration, and structured output generation. By combining LLaMA 3.3-70b's language understanding capabilities with a CrewAI sequential multi-agent pipeline, the system transforms high-level natural language marketing goals into validated, dependency-tracked execution schedules ready for real-world use.

The project provided practical exposure to several key technical disciplines: full-stack web development with React and FastAPI, multi-agent AI system design with CrewAI, LLM prompt engineering for structured outputs, fault-tolerant text parsing strategies, OAuth 2.0 authentication with Firebase, and cloud deployment on Vercel and Render.

A key design insight gained during development was the superiority of structured plain-text prefixes (TASK/DESC/COST etc.) over JSON for LLM responses — LLMs frequently produce malformed JSON, whereas a line-by-line prefix parser is significantly more robust. Similarly, the choice of Process.sequential for the CrewAI crew was essential, as the strict data dependencies between agents make parallel execution infeasible.

The system architecture is well-suited for production enhancement: the mock validation tools are designed as drop-in replacements for real API calls, the stateless FastAPI backend supports horizontal scaling, and the modular agent design allows individual agents to be upgraded independently.

Future Scope & Enhancements

- Persistent Database Integration: Introduce PostgreSQL or MongoDB to persist generated plans, enabling users to retrieve and compare historical schedules. Migration scripts would be managed via Alembic.
- Server-Side JWT Validation: Add Firebase JWT token validation to the FastAPI middleware for production-grade security, eliminating the current client-side-only authentication model.
- API Response Caching: Implement Redis or an in-memory LRU cache keyed by normalized goal string with a 1-hour TTL to reduce Groq API calls and improve response times for repeated goals.
- Real Resource Validation APIs: Replace mock Python dictionaries in Budget Checker, Team Availability, and Timeline Validator with live integrations to CRM systems, HR databases, and project management tools.
- Vector Database for Goal Similarity: Integrate a vector database (e.g., Pinecone or Weaviate) to retrieve similar historical plans, enabling the Planner Agent to generate context-aware, company-specific subtasks.
- Rate Limiting and IP Throttling: Add rate limiting per user IP address to the FastAPI middleware to prevent abuse and protect the Groq API quota.
- Paid Hosting Upgrade: Migrate from Render's free tier to a paid always-on instance to eliminate the 30-50 second cold start latency.
- Horizontal Scaling: Add load balancing and multiple backend instances to support 100+ concurrent users in production.

- GDPR Compliance: Add data consent flows, right-to-erasure functionality, and privacy policy enforcement if the application is deployed to users in the European Union.

- Export Functionality: Allow users to export generated plans as PDF or CSV for sharing with stakeholders outside the application.

References

Reference	URL	Purpose
FastAPI Documentation	https://fastapi.tiangolo.com	REST API framework
CrewAI Documentation	https://docs.crewai.com	Multi-agent orchestration framework
Groq API Documentation	https://console.groq.com/docs	LLM inference API
LLaMA 3.3 Model Card	https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct	Language model reference
LiteLLM Documentation	https://docs.litellm.ai	LLM provider routing for CrewAI
React Documentation	https://react.dev	Frontend UI framework
Firebase Authentication	https://firebase.google.com/docs/auth	User authentication service
Vite Documentation	https://vitejs.dev	Frontend build tool
Tailwind CSS	https://tailwindcss.com/docs	Utility-first CSS framework

React Router	https://reactrouter.com	Client-side routing library
Vercel Documentation	https://vercel.com/docs	Frontend hosting platform
Render Documentation	https://render.com/docs	Backend hosting platform
Pydantic Documentation	https://docs.pydantic.dev	Python data validation